

Week 12 – Functions & Modularity

■ Learning Objectives

- Understand how to define and call Python functions.
- Use parameters and return statements effectively.
- Apply recursion to simple problems.
- Learn modular design for better readability and testing.
- Practice debugging and code organization.

■ Key Concepts

1. Functions encapsulate reusable code.
2. Use def keyword to define a function.
3. Parameters are inputs; return outputs results.
4. Recursion calls a function within itself with a base case.
5. Modular design splits programs into logical components.

Example 1 – Function Definition and Call

```
def greet(name):  
    print("Hello,", name)  
  
greet("Alice")
```

Example 2 – Return Values

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print("Sum =", result)
```

Example 3 – Factorial (Recursion)

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n-1)  
  
print(factorial(5)) # Output: 120
```

Example 4 – Fibonacci (Recursion)

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)  
  
for i in range(6):  
    print(fib(i), end=" ")
```

Example 5 – Modular Design

```
def read_numbers():  
    return list(map(int, input("Enter numbers: ").split()))  
  
def average(nums):  
    return sum(nums)/len(nums)  
  
def main():  
    data = read_numbers()  
    print("Average =", average(data))  
  
main()
```

■ Practice Problems

Practice 1 – Maximum of Three

```
def maximum(a, b, c):  
    return max(a, b, c)  
  
print(maximum(4, 9, 2))
```

Practice 2 – Power Function

```
def power(base, exp):  
    result = 1  
    for _ in range(exp):  
        result *= base  
    return result  
  
print(power(2, 5)) # Output: 32
```

Practice 3 – Count Words in Text

```
def count_words(text):  
    words = text.split()  
    return len(words)  
  
print(count_words("Functions make code reusable"))
```

■ Debugging Tips

- Use `print()` to trace variable values.
- Always check your base case in recursion.
- Start testing with small inputs.
- Use clear and descriptive function names.
- Keep functions short (1 purpose per function).

■ Homework Challenge

Write a program with three functions: (1) read numbers, (2) calculate median, (3) display result.

```
def read_numbers():
    return list(map(int, input("Enter numbers: ").split()))

def median(nums):
    nums.sort()
    n = len(nums)
    mid = n // 2
    if n % 2 == 0:
        return (nums[mid-1] + nums[mid]) / 2
    return nums[mid]

def main():
    data = read_numbers()
    print("Median =", median(data))

main()
```